

Laboratório L1: One-Time Pad de Shannon

Profa. Michele Nogueira

Data:	18 de agosto de 2026 (não serão aceitas entregas atrasadas)
Modalidade:	aula online assíncrona
Duração prevista:	100 minutos
Organização:	atividade individual
Evidência:	Relatório L1 e código-fonte
Pontuação:	Max. 10 pontos

Questão norteadora: por que uma cifra matematicamente perfeita pode se tornar insegura quando a chave é gerada, armazenada, transmitida ou reutilizada de forma inadequada?

1 Objetivos de aprendizagem

Ao final deste laboratório, o(a) estudante deverá ser capaz de:

- explicar o funcionamento da cifra *One-Time Pad* (OTP) e as condições necessárias para o sigilo perfeito de Shannon;
- implementar corretamente a cifração e a decifração por meio da operação XOR;
- empregar uma fonte criptograficamente segura para gerar chaves;
- demonstrar experimentalmente por que uma chave OTP jamais deve ser reutilizada;
- analisar erros de uso e desafios de proteção, distribuição e descarte de chaves em redes.

2 Fundamentação

Considere uma mensagem M , uma chave K e um texto cifrado C , todos representados por sequências de bits de mesmo comprimento. A OTP utiliza a operação XOR ($\oplus$):

$$\begin{aligned}C &= M \oplus K, \\M &= C \oplus K.\end{aligned}$$

Isso funciona porque $(M \oplus K) \oplus K = M$. A OTP oferece sigilo perfeito quando, simultaneamente:

1. a chave é verdadeiramente aleatória;
2. a chave tem o mesmo comprimento da mensagem;

3. a chave permanece secreta;
4. cada chave é utilizada uma única vez; e
5. a chave é descartada de forma segura após o uso.

Se duas mensagens forem cifradas com a mesma chave, então:

$$C_1 \oplus C_2 = (M_1 \oplus K) \oplus (M_2 \oplus K) = M_1 \oplus M_2.$$

A chave é cancelada, produzindo uma relação entre os textos em claro. Essa falha, conhecida como *two-time pad*, pode permitir a recuperação parcial ou total das mensagens.

3 Ambiente e arquivos

Use Python 3 em um ambiente local ou no Google Colab. Trabalhe somente com os arquivos sintéticos fornecidos para a disciplina ou com mensagens criadas por você especificamente para esta atividade. Não utilize dados pessoais, credenciais ou conteúdo confidencial real.

Crie um arquivo chamado `otp.py`. Você poderá usar apenas módulos da biblioteca-padrão do Python, incluindo `secrets`, `os`, `pathlib` e `base64`.

Atenção

Não use `random` para gerar chaves criptográficas. Prefira `secrets.token_bytes(n)` ou `os.urandom(n)`. Execute o XOR sobre `bytes`, não diretamente sobre caracteres. Se a entrada for texto, converta-a com UTF-8 antes da cifração e faça a conversão inversa somente depois da decifração.

4 Atividades práticas

4.1 Parte A – Implementação básica (30 minutos)

Implemente, sem usar bibliotecas criptográficas prontas, as seguintes funções:

```
1 def gerar_chave(tamanho: int) -> bytes:
2     """Gera uma chave criptograficamente segura com tamanho bytes."""
3
4 def xor_bytes(dados: bytes, chave: bytes) -> bytes:
5     """Calcula o XOR entre sequencias de mesmo comprimento."""
6
7 def cifrar(mensagem: bytes, chave: bytes) -> bytes:
8     """Cifra mensagem usando OTP."""
9
10 def decifrar(cifrado: bytes, chave: bytes) -> bytes:
11     """Decifra um texto cifrado usando OTP."""
```

Sua implementação deverá rejeitar entradas incompatíveis e apresentar uma mensagem de erro clara quando os comprimentos forem diferentes. Em seguida:

1. escolha uma mensagem de teste com pelo menos 40 caracteres;
2. converta-a para **bytes** usando UTF-8;
3. gere uma chave com exatamente o mesmo número de bytes;
4. cifre e decifre a mensagem;
5. apresente mensagem, chave e texto cifrado em hexadecimal;
6. verifique programaticamente que a mensagem recuperada é idêntica à original.

Inclua pelo menos três testes: texto ASCII, texto UTF-8 com acentos e uma sequência binária sintética. Explique por que o número de caracteres de uma *string* pode ser diferente do número de bytes em UTF-8.

4.2 Parte B – Cifrando um arquivo sintético (20 minutos)

Implemente a leitura e a cifração de um pequeno arquivo sintético fornecido. Salve os seguintes resultados:

- `arquivo.cifrado`, contendo somente os bytes cifrados;
- `arquivo.recuperado`, produzido pela decifração;
- a chave em arquivo separado, apenas durante o experimento.

Compare o arquivo original e o recuperado byte a byte. Registre o tamanho dos três arquivos e informe se a igualdade foi confirmada. Ao terminar, remova a cópia local da chave do diretório de entrega e explique por que armazenar chave e texto cifrado no mesmo local reduz a segurança.

4.3 Parte C – Falha crítica: reuso de chave (30 minutos)

Crie duas mensagens sintéticas, M_1 e M_2 , com o mesmo comprimento em bytes. Gere uma única chave K e, **deliberadamente apenas para o experimento**, reutilize-a:

$$C_1 = M_1 \oplus K \quad \text{e} \quad C_2 = M_2 \oplus K.$$

Realize e documente os passos abaixo:

1. calcule $C_1 \oplus C_2$ e confirme que o resultado é igual a $M_1 \oplus M_2$;
2. assuma que um atacante conhece M_1 e C_1 e recupere a chave por $K = M_1 \oplus C_1$;
3. use a chave recuperada para obter M_2 a partir de C_2 ;
4. explique por que o experimento viola uma condição essencial da OTP;
5. repita o teste com chaves independentes e mostre por que o mesmo procedimento deixa de recuperar M_2 .

4.4 Parte D – Análise de uso em redes (20 minutos)

Responda de forma fundamentada:

1. Por que a OTP oferece confidencialidade, mas não garante, isoladamente, integridade, autenticidade ou não repúdio? Apresente um exemplo de alteração maliciosa no texto cifrado.
2. Quais dificuldades surgiriam para distribuir chaves OTP entre dois pontos de uma rede que trocam grande volume de dados?
3. Quais riscos existem nas etapas de geração, transporte, armazenamento, cópia de segurança e descarte das chaves?
4. Em que a OTP difere conceitualmente de uma cifra de fluxo moderna, apesar de ambas combinarem dados com um fluxo de bits por XOR?
5. Proponha pelo menos quatro controles para proteger o ciclo de vida das chaves.

5 Relatório L1

Produza um relatório em PDF, com extensão sugerida de 3 a 5 páginas, contendo:

1. identificação do(a) estudante;
2. descrição objetiva da implementação e das decisões adotadas;
3. evidências dos testes das Partes A e B;
4. demonstração algébrica e experimental do reuso de chave na Parte C;
5. respostas da Parte D;
6. conclusão relacionando os resultados à questão norteadora;
7. referências consultadas, se houver.

As evidências devem incluir trechos relevantes da execução, valores em hexadecimal e uma breve interpretação. Não é necessário inserir capturas de tela de todo o código. O código completo deverá ser entregue separadamente.

Entrega obrigatória

Entregue um arquivo compactado chamado `L1_NomeSobrenome.zip`, contendo apenas:

- `relatorio_L1.pdf`;
- `otp.py`;
- os arquivos sintéticos de entrada e os resultados solicitados;
- opcionalmente, um `README.txt` com instruções de execução.

Não inclua chaves reais, credenciais, dados pessoais ou outros segredos no arquivo de entrega.

6 Critérios de avaliação

Critério	Peso
Correção da implementação, validação de tamanhos e uso seguro da fonte de aleatoriedade	30%
Testes, cifração do arquivo e evidências de correção	20%
Demonstração e interpretação do ataque por reuso de chave	25%
Análise de segurança, proteção de chaves e relação com redes	20%
Clareza, organização e reprodutibilidade da entrega	5%

7 Regras acadêmicas e de segurança

- A atividade é individual. Discussões conceituais são permitidas, mas código, experimentos e texto do relatório devem ser autorais.
- Não utilize ferramentas de IA generativa.
- Não execute experimentos contra sistemas, redes ou dados de terceiros.
- O código deverá ser legível, comentado e executável em Python 3 sem dependências externas.